Object-Oriented Software Engineering
Using UML, Patterns, and Java

Chapter 8, Object Design: Object Constraint Language



Outline of the Lecture

- OCL
- Simple predicates
- Preconditions
- Postconditions
- Contracts
- Sets, Bags, and Sequences

OCL: Object Constraint Language

- Formal language for expressing *constraints* over a set of objects and their attributes in a model
  - Used to write constraints that cannot otherwise be expressed in a diagram
- Based on Sets and Multi Sets.

OCL Basic Concepts

- OCL expressions
  - Return **True** or **False**
  - Are evaluated in a specified context, either a class or an operation
  - All constraints apply to all instances.

OCL Simple Predicates

Example:

```
context Tournament inv:
  self.getMaxNumPlayers() > 0
```

In English:

"The maximum number of players in any tournament should be a postive number."

Notes:

- "self" denotes all instances of "Tournament"
- OCL uses the same dot notation as Java.

OCL Preconditions

Example:

```
context Tournament::acceptPlayer(p) pre:
  not self.isPlayerAccepted(p)
```

In English:

"The acceptPlayer(p) operation can only be invoked if player p has not yet been accepted in the tournament."

Notes:

- The context of a precondition is an operation
- isPlayerAccepted(p) is an operation defined by the class Tournament.

OCL Postconditions

Example:

```
context Tournament::acceptPlayer(p) post:
  self.getNumPlayers() =
     self@pre.getNumPlayers() + 1
```

In English:

"The number of accepted player in a tournament increases by one after the completion of acceptPlayer()"

Notes:

- self@pre denotes the state of the tournament before the invocation of the operation.
- Self denotes the state of the tournament after the completion of the operation.

OCL Contract for acceptPlayer() in Tournament

```
context Tournament::acceptPlayer(p) pre:
  not isPlayerAccepted(p)

context Tournament::acceptPlayer(p) pre:
  getNumPlayers() < getMaxNumPlayers()

context Tournament::acceptPlayer(p) post:
  isPlayerAccepted(p)

context Tournament::acceptPlayer(p) post:
  getNumPlayers() = @pre.getNumPlayers() + 1
```

OCL Contract for removePlayer() in Tournament

```
context Tournament::removePlayer(p) pre:
  isPlayerAccepted(p)

context Tournament::removePlayer(p) post:
  not isPlayerAccepted(p)

context Tournament::removePlayer(p) post:
  getNumPlayers() = @pre.getNumPlayers() - 1
```

Java Implementation of Tournament class (Contract as a set of JavaDoc comments)

```
public class Tournament {
/** The maximum number of players
 * is positive at all times.
 * @invariant maxNumPlayers > 0
 */
private int maxNumPlayers;

/** The players List contains
 *  references to Players who are
 *  are registered with the
 *  Tournament. */
private List players;

/** Returns the current number of
 * players in the tournament. */
public int getNumPlayers() {…}

/** Returns the maximum number of
 * players in the tournament. */
public int getMaxNumPlayers() {…}
```

```
/** The acceptPlayer() operation
 * assumes that the specified
 * player has not been accepted
 * in the Tournament yet.
 * @pre !isPlayerAccepted(p)
 * @pre getNumPlayers()<maxNumPlayers
 * @post isPlayerAccepted(p)
 * @post getNumPlayers() =
 *      @pre.getNumPlayers() + 1
 */
public void acceptPlayer (Player p) {…}

/** The removePlayer() operation
 * assumes that the specified player
 * is currently in the Tournament.
 * @pre isPlayerAccepted(p)
 * @post !isPlayerAccepted(p)
 * @post getNumPlayers() =
 *      @pre.getNumPlayers() - 1
 */
public void removePlayer(Player p) {…}

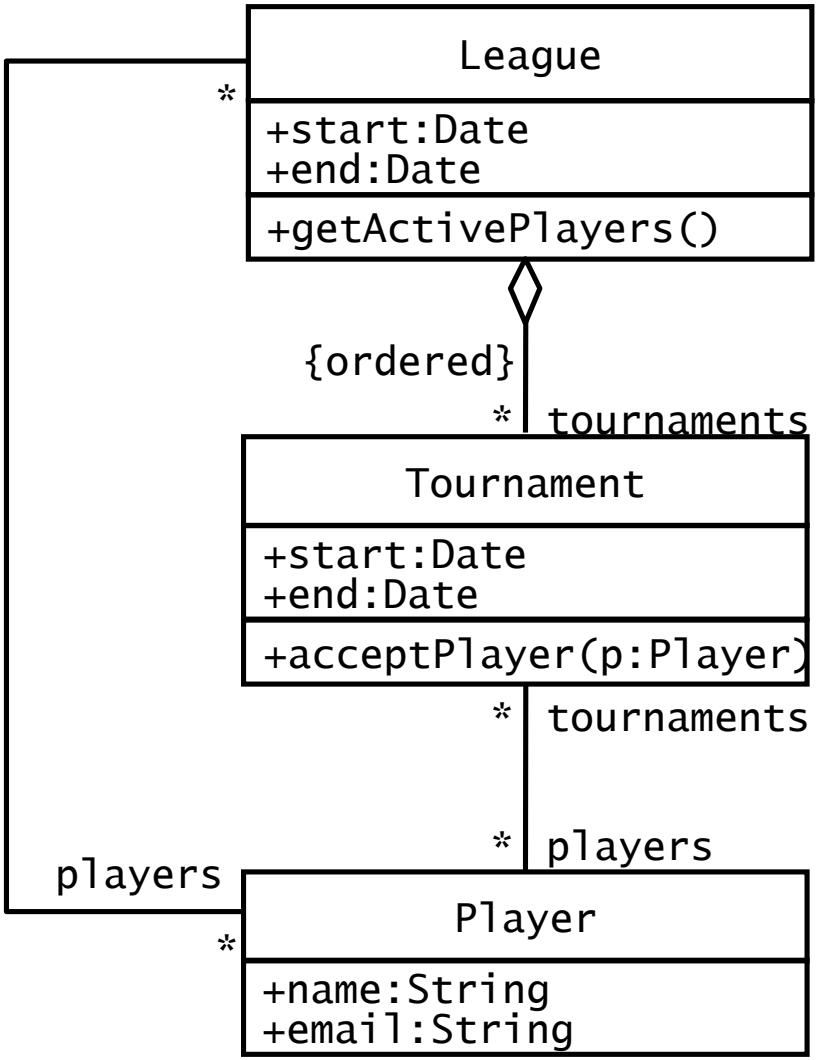
}
```

Constraints can involve more than one class

How do we specify constraints on on a group of classes?

Starting from a specific class in the UML class diagram, we navigate the associations in the class diagram to refer to the other classes and their properties (attributes and Operations).

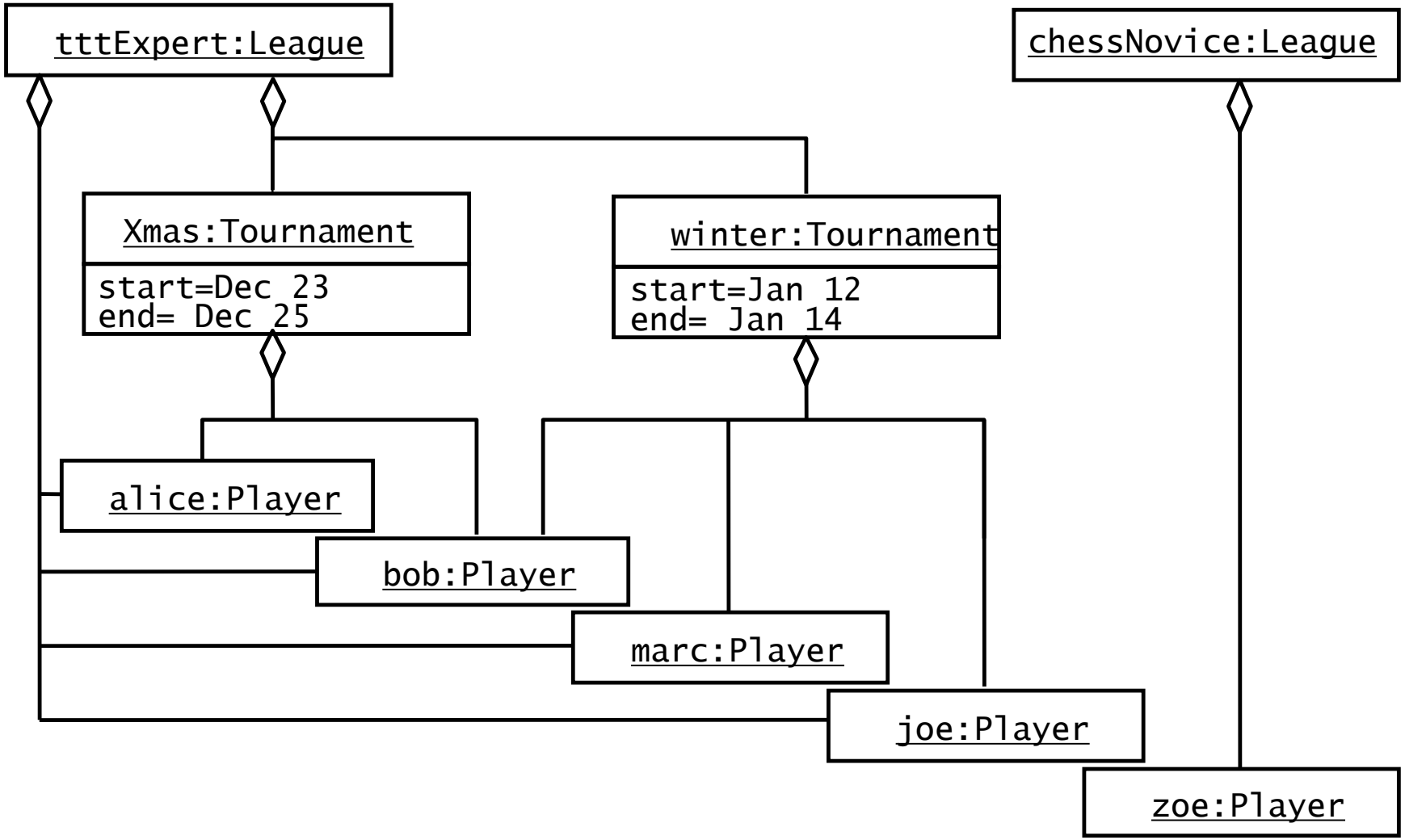
Example from ARENA: League, Tournament and Player



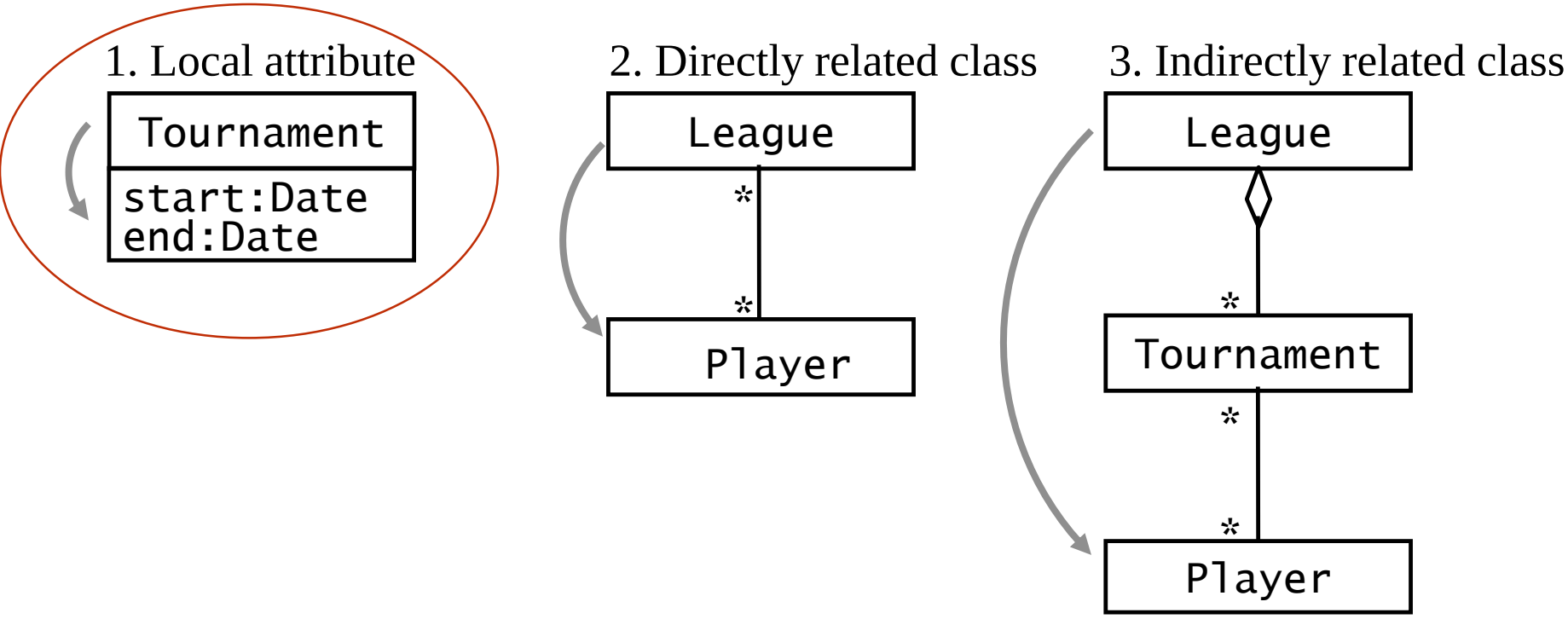
Constraints:

1. A Tournament's planned duration must be under one week.
2. Players can be accepted in a Tournament only if they are already registered with the corresponding League.
3. The number of active Players in a League are those that have taken part in at least one Tournament of the League.

Instance Diagram: 2 Leagues, 5 Players, 2 Tournaments



3 Types of Navigation through a Class Diagram



Any constraint for an arbitrary UML class diagram can be specified using only a combination of these 3 navigation types!

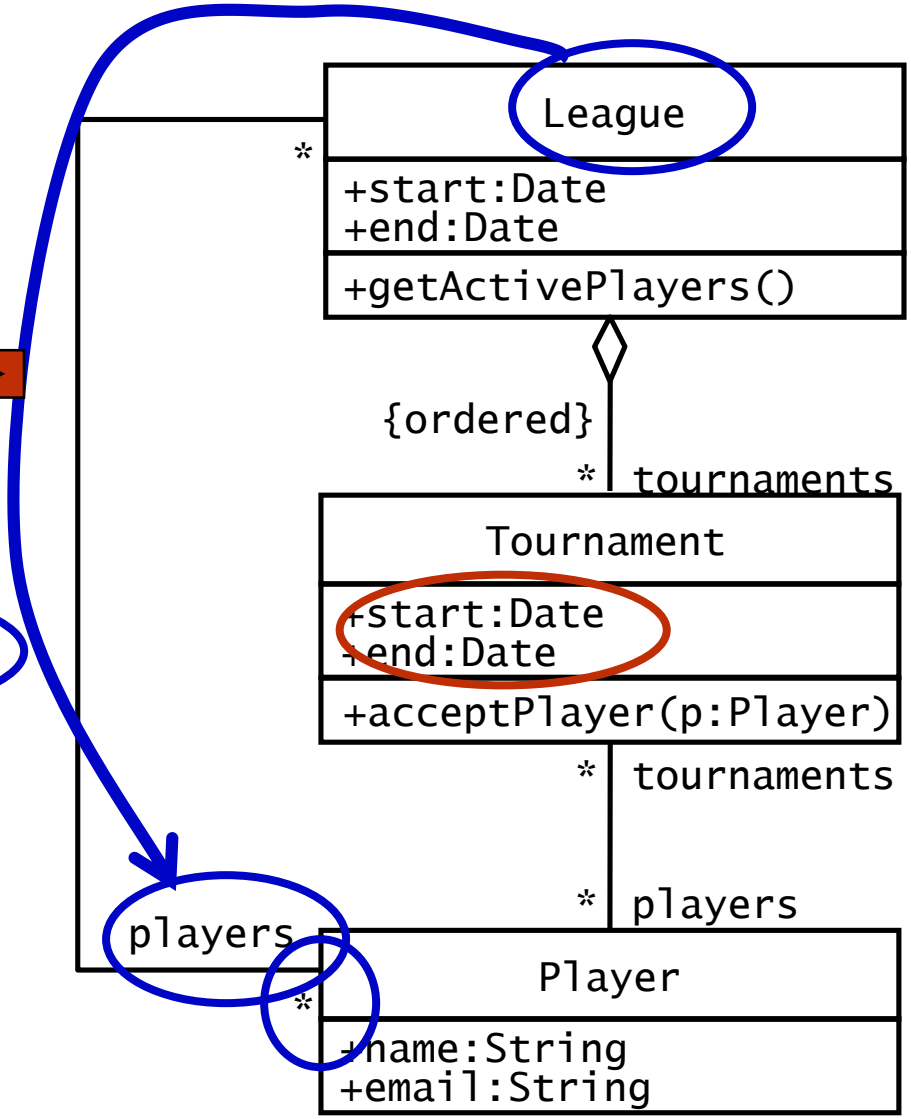
Specifying the Model Constraints in OCL

Local attribute navigation

```
context Tournament inv:
  end - start <= 7
```

Directly related class navigation

```
context
 Tournament::acceptPlayer(p)
  pre:
  league.players->includes(p)
```



OCL-Collection

- The OCL-Type Collection is the generic superclass of a collection of objects of Type T
- Subclasses of Collection are
  - Set: Set in the mathematical sense. Every element can appear only once
  - Bag: A collection, in which elements can appear more than once (also called multiset)
  - Sequence: A multiset, in which the elements are ordered
- Example for Collections:
  - Set(Integer): a set of integer numbers
  - Bag(Person): a multiset of persons
  - Sequence(Customer): a sequence of customers

OCL Sets, Bags and Sequences

- Sets, Bags and Sequences are predefined in OCL and subtypes of Collection. OCL offers a large number of predefined operations on collections. They are all of the form:

```
collection->operation(arguments)
```

OCL-Operations for OCL-Collections (1)

`size: Integer`

Number of elements in the collection

`includes(o:OclAny): Boolean`

True, if the element `o` is in the collection

`count(o:OclAny): Integer`

Counts how many times an element is contained in the collection

`isEmpty: Boolean`

True, if the collection is empty

`notEmpty: Boolean`

True, if the collection is not empty

The OCL-Type **OclAny** is the most general OCL-Type.

OCL-Operations for OCL-Collections(2)

`union(c1:Collection)`
Union with collection **`c1`**

`intersection(c2:Collection)`
Intersection with Collection **`c2`** (contains only elements, which appear in the collection as well as in collection **`c2`** auftreten)

`including(o:OclAny)`
Collection containing all elements of the Collection and element **`o`**

`select(expr:OclExpression)`
Subset of all elements of the collection, for which the OCL-expression **`expr`** is true.

How do we get OCL-Collections?

- A collection can be generated by explicitly enumerating the elements from the UML model
- A collection can be generated by navigating along one or more 1-N associations in the UML model
  - Navigation along a single 1:n association yields a Set
  - Navigation along a couple of 1:n associations yields a Bag (Multiset)
  - Navigation along a single 1:n association labeled with the constraint {ordered} yields a Sequence

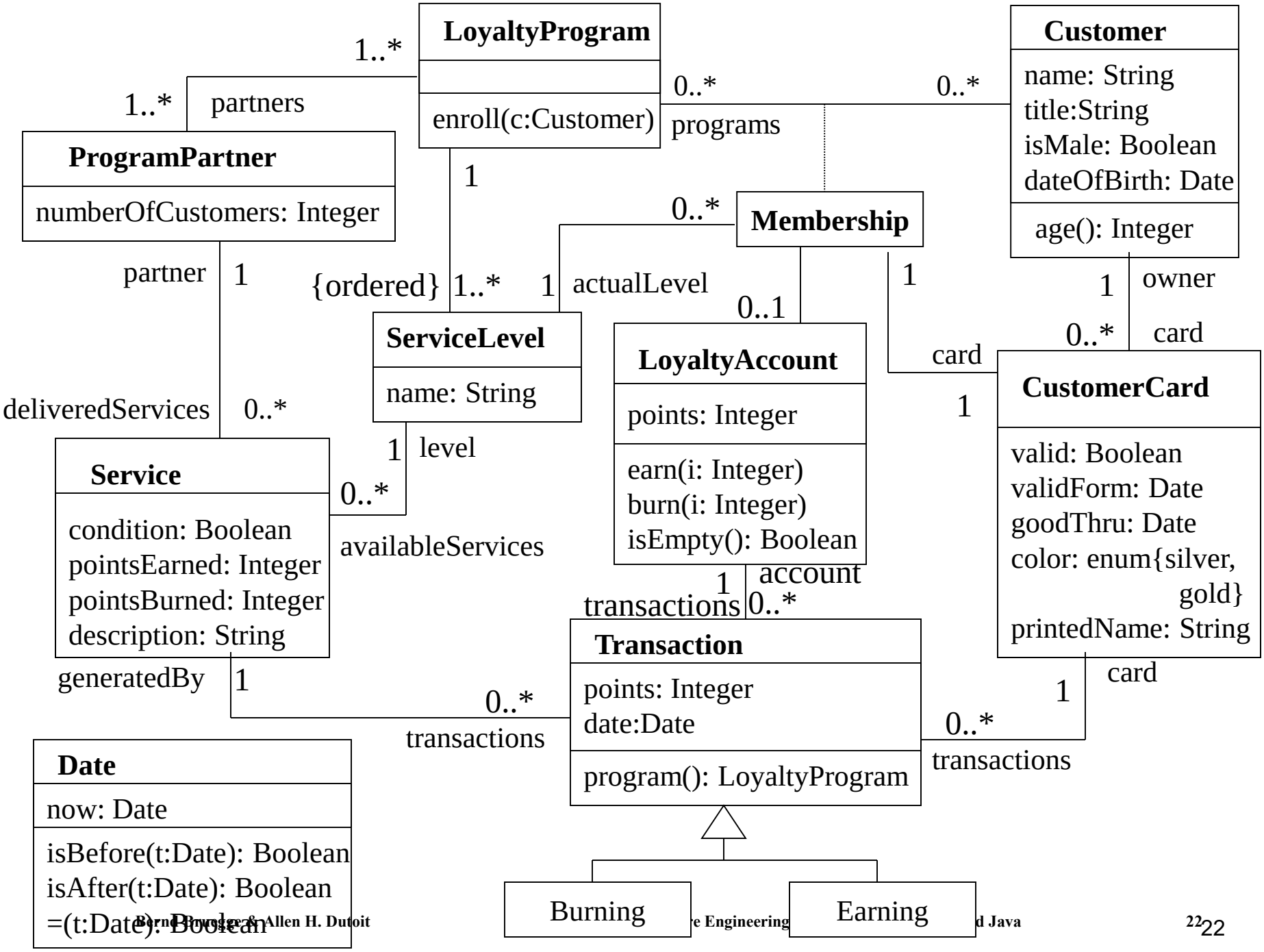
Loyalty Program Problem Statement

A customer loyalty program (LoyaltyProgram) associated many program partners (ProgramPartner) that offer different kinds of bonuses called services (Service) with customers. A customer (Customer) can enter a loyalty program by filling out a form and obtainining a customer card (CustomerCard). A customer can enter many loyalty programs. A customer can have multiple cards. A customer card is uniquely associated with a single customer

Each customer card is associated with a loyalty account (LoyaltyAccount) and characterized by a service level (ServiceLevel). The loyalty account allows customers to save bonus points in their account. The basic service level is silver. Customers who make a lot of transactions get a higher level of service, gold or platinum. Based on the service level and the saved bonus points, customers can "buy" services from a program partner for a specific number of points. So, earning and Buying points are types of transactions (Transaction) on a loyalty account involving services provided by the program partners. They are always only performed for a single customer card.

- **Source: Jos Warmer& Anneke Kleppe,**

  The Object Constraint Language: Getting your models ready for MDA, 2nd edition, Addison Wesley, 2003.

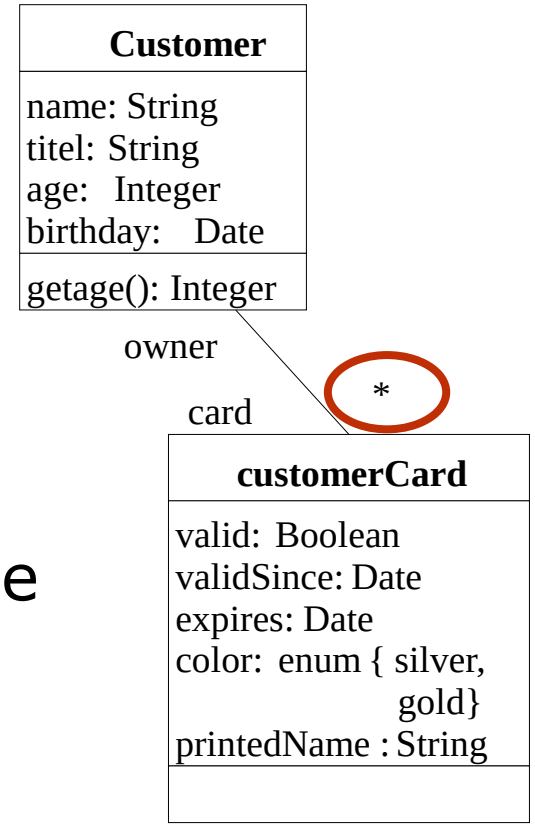

LoyaltyProgram
enroll(c:Customer)
Customer
name: String
title:String
isMale: Boolean
dateOfBirth: Date
age(): Integer
1..*
partners
1..*
programs
0..*
0..*
ProgramPartner
numberOfCustomers: Integer
partner
1
1
{ordered}
1..*
1
actualLevel
0..*
Membership
1
owner
1
0..*
card
ServiceLevel
name: String
LoyaltyAccount
points: Integer
earn(i: Integer)
burn(i: Integer)
isEmpty(): Boolean
0..1
card
1
CustomerCard
valid: Boolean
validForm: Date
goodThru: Date
color: enum{silver, gold}
printedName: String
deliveredServices
0..*
1
level
Service
condition: Boolean
pointsEarned: Integer
pointsBurned: Integer
description: String
0..*
availableServices
1
account
transactions
0..*
generatedBy
1
0..*
transactions
Transaction
points: Integer
date:Date
program(): LoyaltyProgram
1
card
0..*
transactions
Date
now: Date
isBefore(t:Date): Boolean
isAfter(t:Date): Boolean
=(t:Date): Boolean
Burning
Earning

Navigation through a 1-to-n-Association (Directly related class navigation)

Example: A Customer should not have more than 4 cards

```
context Customer inv:
  card->size <= 4
```

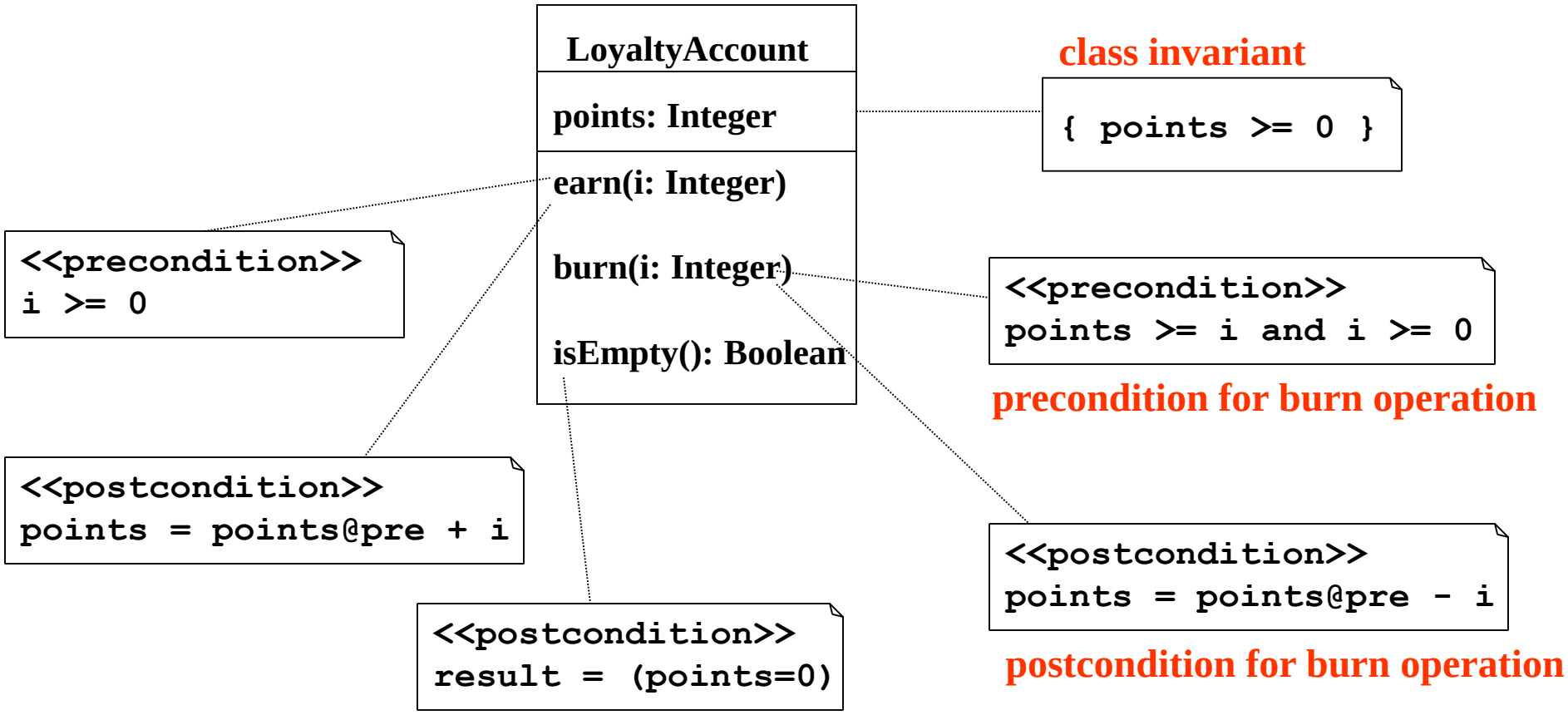
card denotes a set of **customercards**



- Alternative writing style

```
Customer
card->size <= 4
```

OCL Constraints on the Loyalty Account



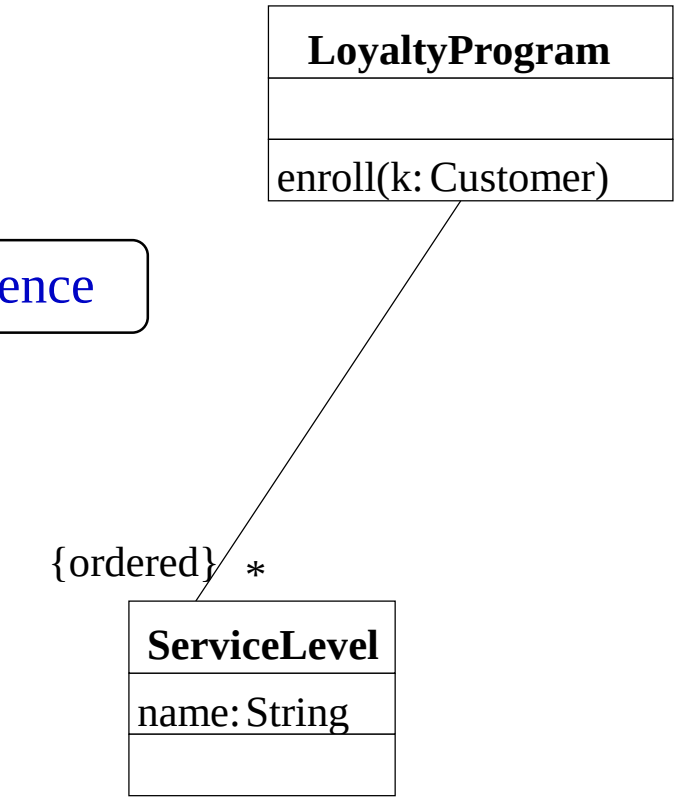
Navigation through a constrained Association

- Navigation through an association with the constraint **{ordered}** yields a *sequence*
- Problem Statement: The number of service levels must be 2

LoyaltyProgram

servicelevel->size = 2

servicelevel denotes a sequence



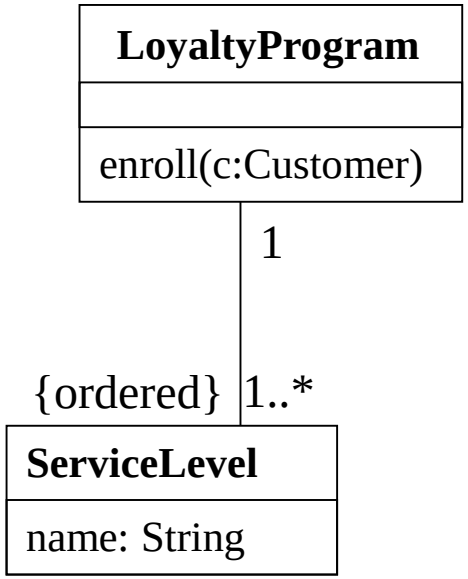
Operations on OCL-Type Sequence

first: T	The first element of a sequence
last: T	The last element of a sequence
at(index:Integer): T	Element **index** in the sequence

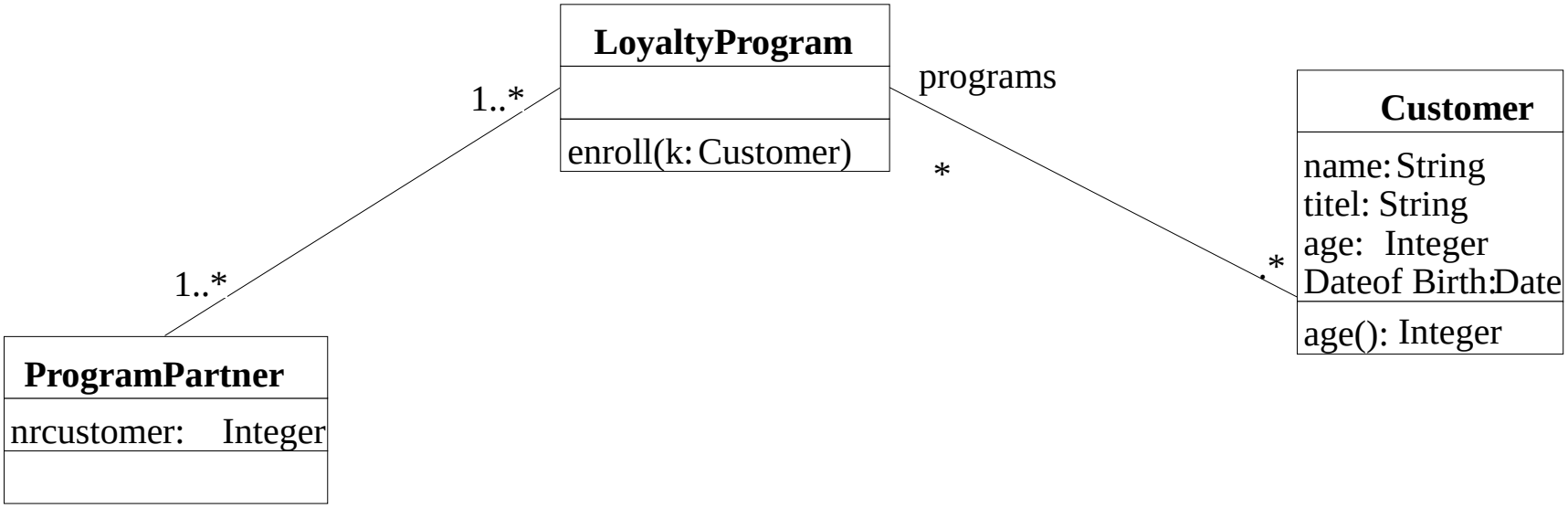
Problemstatement: The first level in the loyalty program has the name 'Silver‘. The highest level you can reach is ‘Platinum'.“

OCL-Invariants:

```
LoyaltyProgram:
servicelevel->first.name = "Silver"
LoyaltyProgram:
servicelevel->last.name = "Platinum"
```



Navigation through several 1:n-Associations



Problem Statement: "The number of customers registered by a program partner must be equal to the number of customers belonging to a loyalty program offered by the program partner"

```
Context ProgramPartner inv
nrcustomer = loyaltyprogram->customer->size
```

loyaltyprogram denotes a set of objects of type L**oyaltyProgram**

customer denotes a multiset of objects of type **Customer**

ProgramPartner

```
nrcustomer = loyaltyprogram->customer->size
```

Conversion between OCL-Collections

- OCL offers operations to convert OCL-Collections:

asSet
  Transforms a multiset or sequence into a set

asBag
  transforms a set or sequence into a multiset

asSequence
  transforms a set or multiset into a sequence.

Example of a Conversion

programPartner

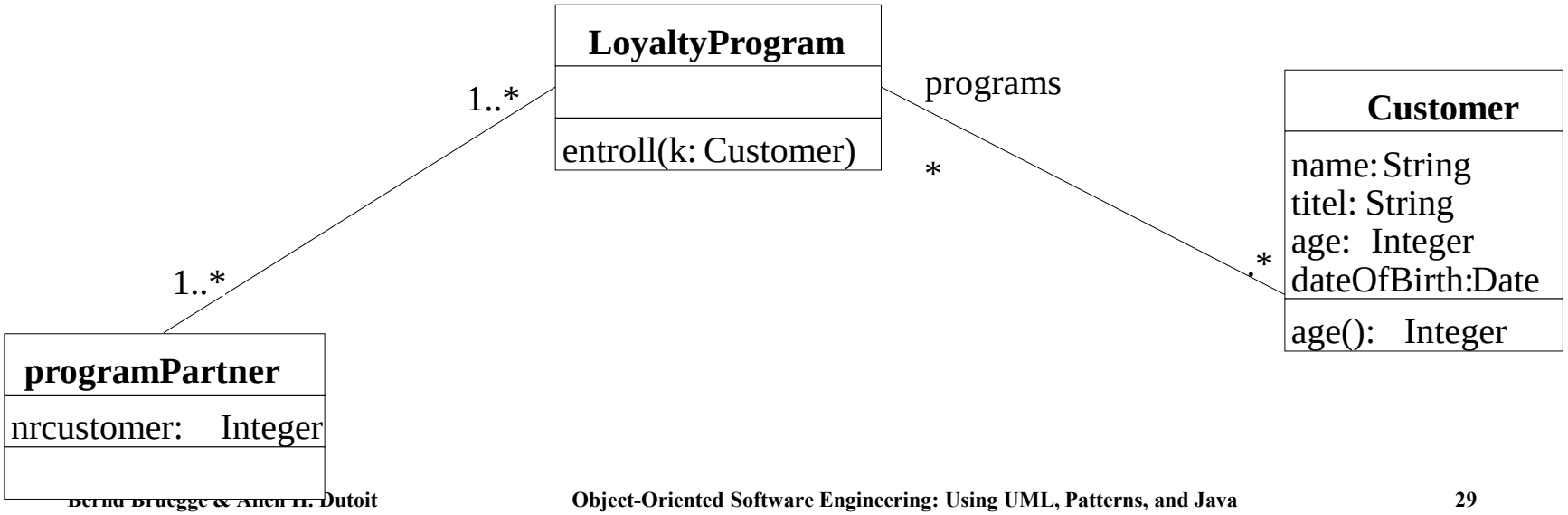
```
nrcustomer = loyaltyprogram->customer->size
```

Caution: This OCL expression may contain a specific customer several times!

We can get the number of unique customers as follows:

programPartner

```
nrcustomer = loyaltyprogram->customer->asSet->size
```

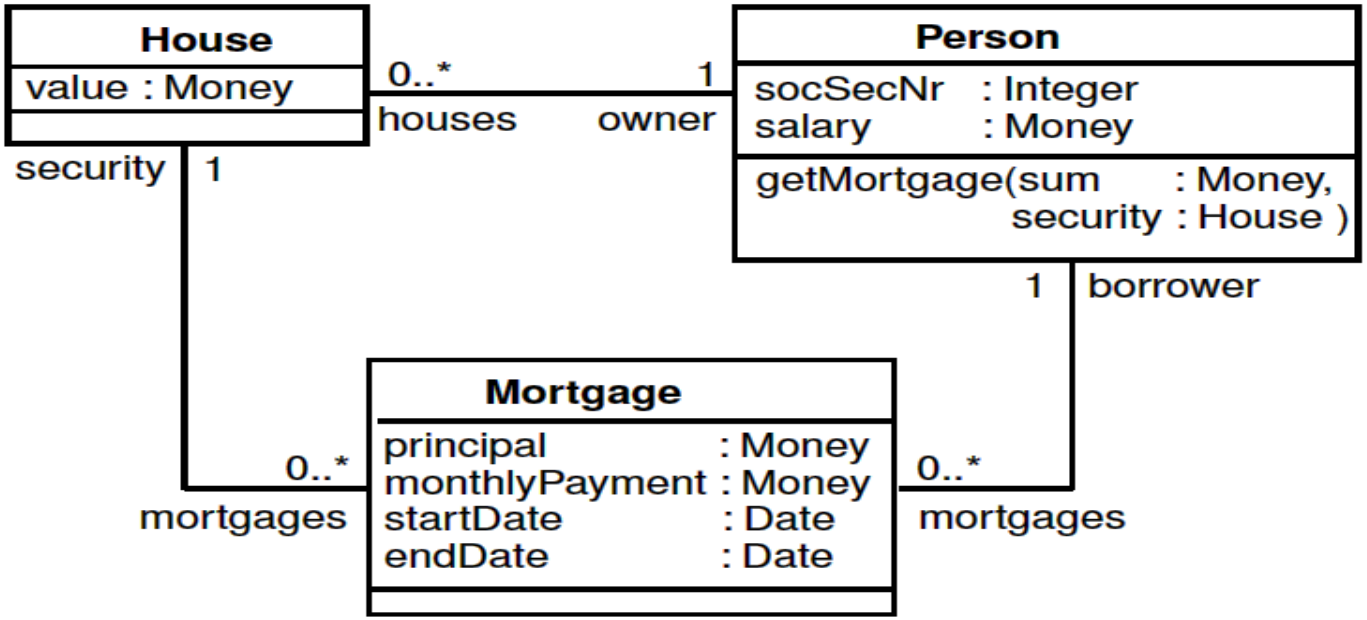


Evaluating OCL Expressions when navigating a UML Class Diagram

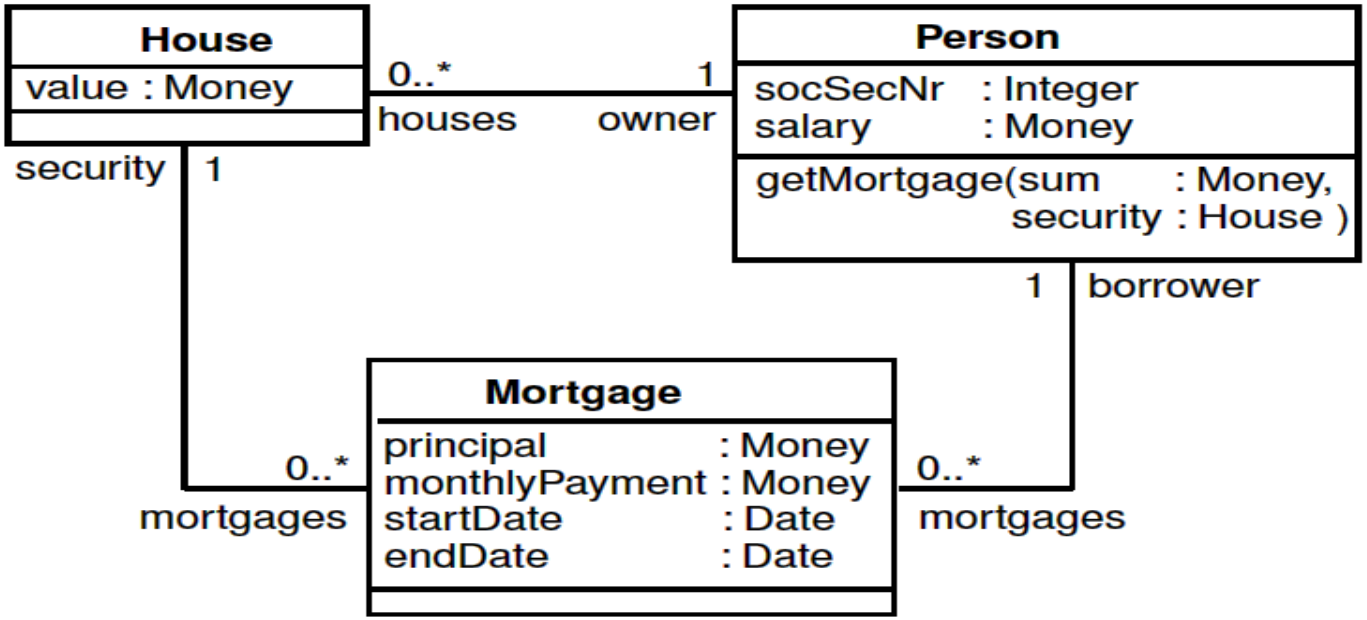
The value of any OCL expression is an object or a collection of objects

- Navigating an association-end with multiplicity 1
  - Result: a single object
- Navigating an association-end with multiplicity 0..1
  - Empty set, if there is no object, otherwise a single object
- Navigating several association-ends with multiplicity n
  - Result: A collection of objects
    - If the association is {ordered}, the navigation result is a OCL sequence
    - Multiple "1-n" associations result in a OCL multiset (bag)
    - Otherwise the navigation result is a OCL set.

Another Navigation Example: A Mortgage System



Another Navigation Example: A Mortgage System

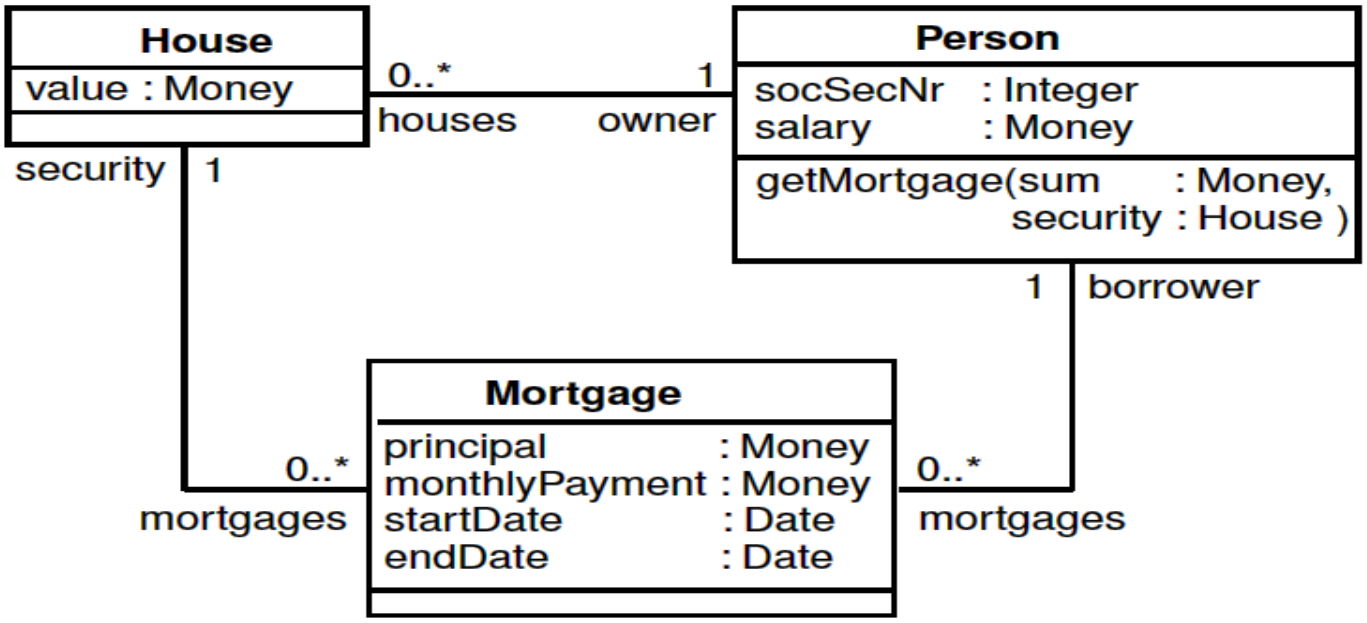


Constraints (in the Problem Statement):

1. "A person can have a mortgage only on the house owned by that person"

2. "The start date of a mortgage must before its end date."

Formal Specification of the Constraints in OCL



1. "A person can have a mortgage only on the house owned by that person"

   context Mortgage **inv**: security.owner = borrower

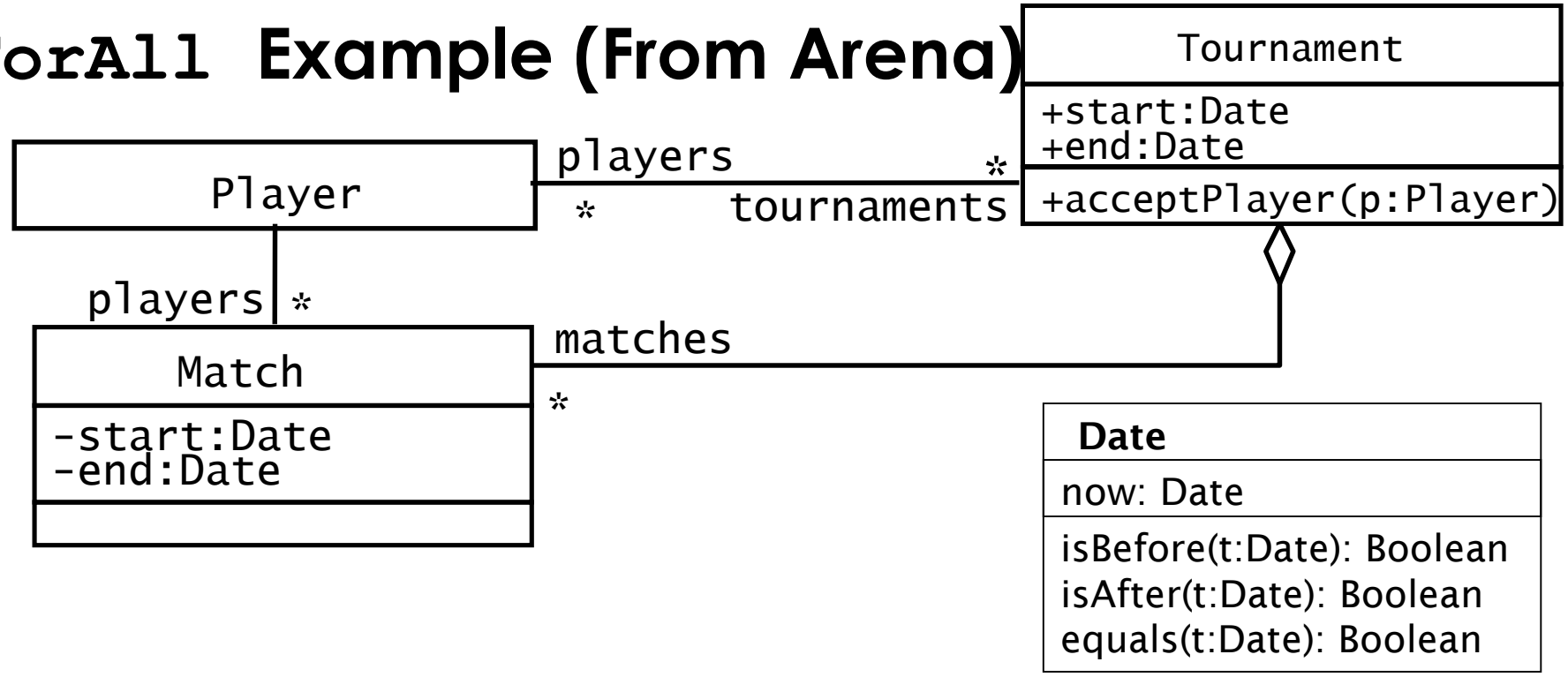
2. "The start date of a mortgage must before its end date."

   context Mortgage **inv**: startDate < endDate

Predicate Calculus vs First-Order Logic

- So far, we have presented OCL as **propositional logic**
  - Propositional logic consists of well formed formulas (wffs) constructed of a set of primitive symbols (**true**, **false**), letters (a, b, x,y,...), and a set of operators (**and** $\wedge$, **or** $\vee$**, not** $\sim$ ...)
- OCL is actually a **first order logic** over the domain of OCL collections
  - Any first order logic also supports quantification
- Existential quantification(in symbolic logic: $\exists$, in OCL: `exists`)
  - The OCL `exists` operator takes a boolean expression as parameter. It evaluates to true if the parameter is true for at least one element of the OCLcollection
- Universal quantification (in symbolic logic:$\forall$, in OCL: `forAll`)
  - The OCL `forAll` operator takes a boolean expression as parameter. It evaluates to true if it is true for all the elements of the OCLcollection.

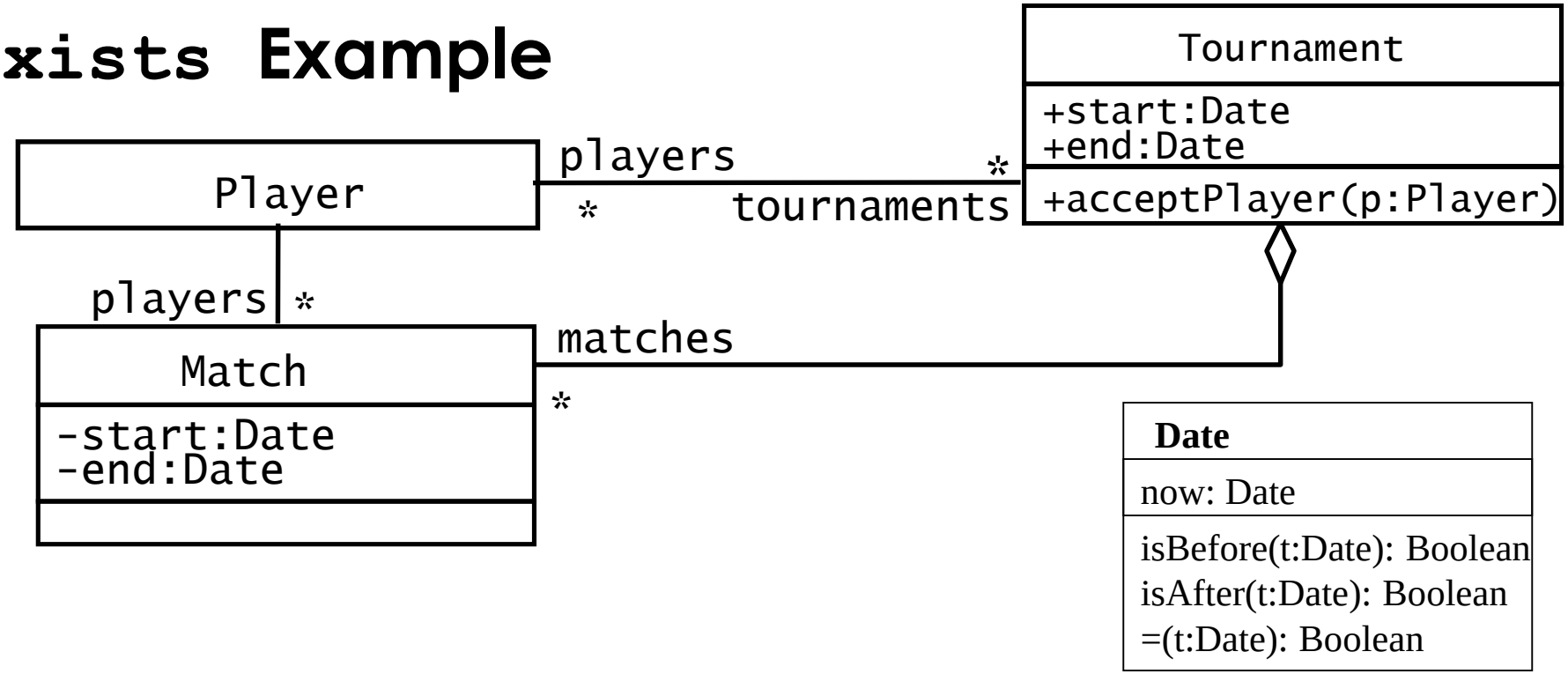
`ForAll` Example (From Arena)



Problem Statement: "All Matches in a Tournament must occur within the time frame of the Tournament"

```
context Tournament inv:
   matches->forAll(m|
      m.start.isAfter(self.start) and
      m.start.isBefore(self.end))
```

Exists Example



Problem Statement: "Each Tournament conducts at least one Match on the first day of the Tournament"

```
context Tournament inv:
   matches->exists(m:Match| m.start.equals(start))
```

Readings about OCL and Tools

- J.B. Warmer, A.G. Kleppe
  - The Object Constraint Language: Getting your Models ready for MDA, Addison-Wesley, 2nd edition, 2003

- The Dresden OCL Toolkit
  - http://dresden-ocl.sourceforge.net/

- The Dresden OCL Toolkit for Eclipse
  - http://sourceforge.net/projects/dresden-ocl/files/dresden-ocl2-for-eclipse/2.0/ocl2-for-eclipse-2.0.tar.gz/download

Readings about Contracts

- B. Meyer

  Object-Oriented Software Construction, 2nd edition, Prentice Hall, 1997.

- B. Meyer,

  Design by Contract: The Lesson of Ariane, Computer, IEEE, Vol. 30, No. 2, pp. 129-130, January 1997. http://archive.eiffel.com/doc/manuals/technology/contract/ariane/page.html

- C. A. R. Hoare,

  An axiomatic basis for computer programming. Communications of the ACM, 12(10):576-585, October 1969. (Good starting point for Hoare logic: http://en.wikipedia.org/wiki/Hoare_logic)

Summary

- Constraints are predicates (often boolean expressions) on UML model elements
- Contracts are constraints on a class that enable class users, implementors and extenders to share the same assumption about the class ("Design by contract")
- OCL is the example of a formal language that allows us to express constraints on UML models
  - The names of the model elements in the UML model are used in the formulation of the OCL predicates
  - The context definition of an OCL expression specifies the model entity for which the OCL expression is defined.
- Complicated constraints involving more than one class, attribute or operation can be formulated by using 3 basic navigation types:
  - Local attribute navigation, directly related class navigation, indirectly related class navigation.

Backup and Additional Slides

